



Основи Python



Programming Language



Created in 1991 by Guido van Rossum



Cross Platform



Freely Usable Even for Commercial Use

Python

«Python простий у використанні, потужний і універсальний, що робить його чудовим вибором як для новачків, так і для експертів».

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello world!");  
    }  
}
```



- Обробка зображень
- Розробка гри
- Наука про дані

```
print("Hello world!")
```



facebook

Google

Dropbox

NETFLIX

Spotify

PHILIPS

Big names using Python

Налаштування Python

Деякі версії Linux постачаються з інстальованим Python. Наприклад, якщо у вас є дистрибутив на основі Red Hat Package Manager (RPM) (наприклад, SUSE, Red Hat, Yellow Dog, Fedora Core та CentOS), ви, імовірно, уже маєте Python у своїй системі, і вам не потрібно робити щось інше.

Залежно від того, яку версію Linux ви використовуєте, версія Python змінюється, і деякі системи не включають програму Interactive DeveLopment Environment (IDLE). Якщо у вас старіша версія Python (2.5.1 або раніша), ви можете встановити новішу версію, щоб мати доступ до IDLE.

Щоб дізнатися, яку версію Python 3 ви встановили, відкрийте командний рядок і запустіть

```
$ python3 --version
```

Якщо ви використовуєте Ubuntu 16.10 або новішу версію, ви можете легко встановити Python 3.8 за допомогою таких команд:

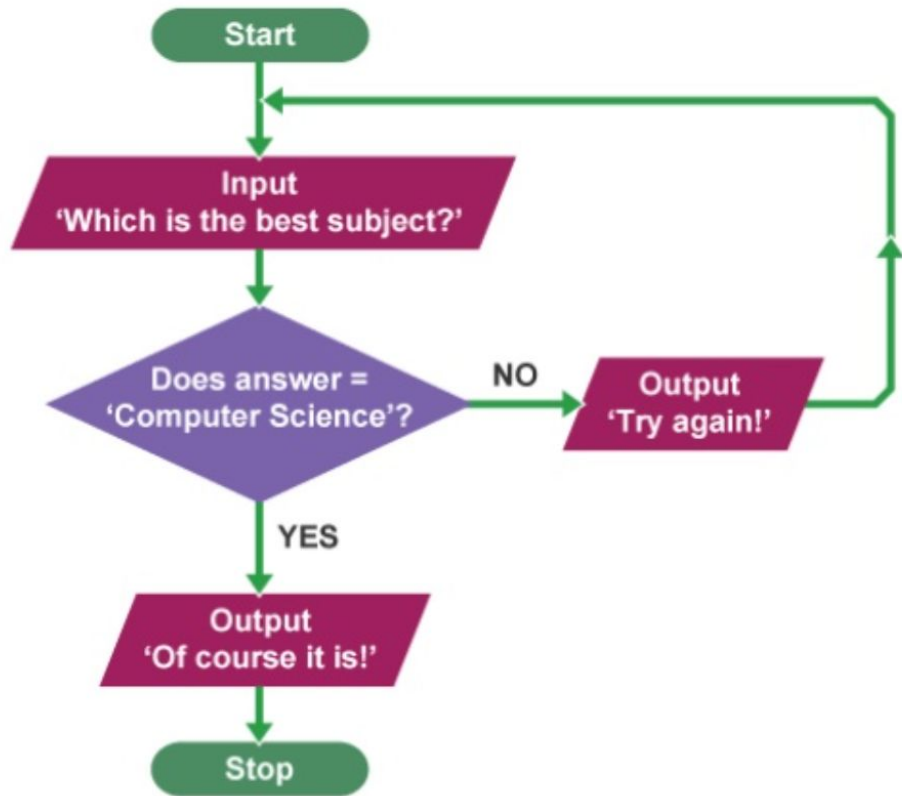
```
$ sudo apt-get update
```

```
$ sudo apt-get install python3.8
```

Змінні та типи даних

- Змінні зберігають і дають імена значенням даних
- Значення даних можуть бути різних типів:
 - int : -5, 0, 1000000
 - float : -2.0, 3.14159
 - bool : True, False
 - str : "Hello world!", "K3WL"
 - list : [1, 2, 3, 4], ["Hello", "world!"], [1, 2, "Hello"], []
 - And many more!
- У Python змінні **не** мають типів!
- Значення даних присвоюються змінним за допомогою "="
`x = 1 # this is a Python comment`
`x = x + 5`
`y = "Python" + " is " + "cool!"`

Умови та цикли



Умови та цикли

- Ядра програмування!
- Покладайтеся на логічні вирази, які повертають **True** або **False**
 - `1 < 2` : **True**
 - `1.5 >= 2.5` : **False**
 - `відповідь == "Computer Science"` : може бути **True** або **False** залежно від значення змінної відповіді
- Логічні вирази можна комбінувати з: *and, or, not*
 - `1 < 2 and 3 < 4` : **True**
 - `1.5 >= 2.5 or 2 == 2` : **True**
 - `not 1.5 >= 2.5` : **True**

Умовні: загальна форма

якщо логічний вираз-1:

код-блок-1

логічний вираз elif-2:

код-блок-2

(скільки завгодно elif)

інше:

код-блок-останній

Умови: Usia SIM (Вік водійських прав)

```
age = 20
```

```
if age < 17:
```

```
    print("Belum bisa punya SIM!")
```

```
else:
```

```
    print("OK, sudah bisa punya SIM.")
```

Умови: використовуйте для введенняUsia SIM

```
age = int(raw_input("Usia: ")) # use input() for Python 3
if age < 17:
    print("Belum bisa punya SIM!")
else:
    print("OK, sudah bisa punya SIM.")
```

Умовні оператори: Оцінювання

```
grade = int(raw_input("Numeric grade: "))  
if grade >= 80:  
    print("A")  
elif grade >= 65:  
    print("B")  
elif grade >= 55:  
    print("C")  
else:  
    print("E")
```

Структура Match-case (з python3.10)

```
grade = int(input("Numeric grade: "))
match grade:
    case g if g >= 80:
        print("A")
    case g if 65 <= g < 80:
        print("B")
    case g if 55 <= g < 65:
        print("C")
    case _:
        print("E")
```

Тернарний оператор у Python

`<expression1> if <condition> else <expression2>`

```
age = int(input("Age: "))  
if age < 18:  
    print("Adult")  
else:  
    print("Minor")
```



```
age = int(input("Age: "))  
print("Adult" if age > 17 else "Minor")
```

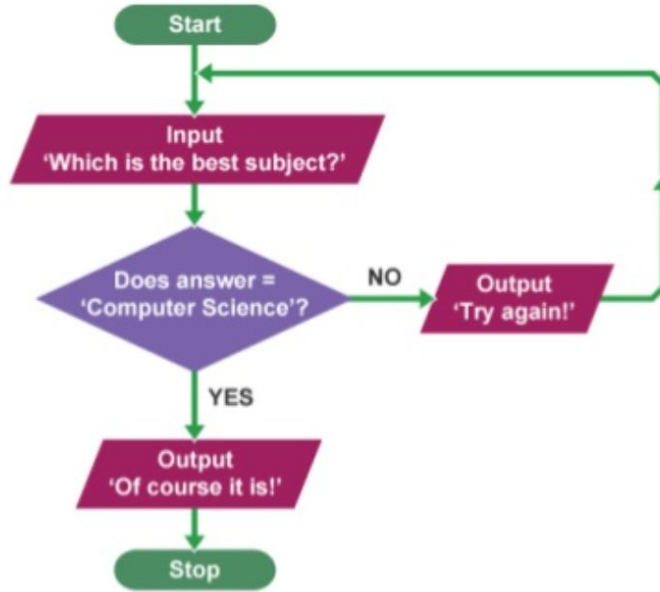
Цикли

- Корисно для повторення коду!
- Два варіанти:

```
while boolean-expression:  
    code-block
```

```
for element in collection:  
    code-block
```

Нескінченні цикли



```
while raw_input("Який предмет найкращий? ") != "Computer Science":  
    print("Try again!")  
print("звісно, цей!")
```

while boolean-expression: code-block

Для циклів

Наразі ми бачили (коротко) два види колекцій:
рядок і список

Для відвідування кожного елемента колекції можна використовувати цикли for:
code-block `for chr in "string": print(chr)`
`for elem in [1,3,5]: print(elem)`

Функції

- інкапсулювати (= membungkus) блоки коду
- Чому функції? Модулярність і повторне використання!
- Ви дійсно бачили приклади функцій:
 - `print()`
 - `raw_input()`
- Родова форма:

```
def function-name(parameters):  
    code-block  
    return value
```

Функції: від Цельсія до Фаренгейта

```
def celsius_to_fahrenheit(celsius):
```

```
    fahrenheit = celsius * 1.8 + 32.0
```

```
    повернутися за Фаренгейтом
```

def назва функції (параметри): значення, що повертається кодовим блоком

Функції: типові та іменовані параметри

```
def hello(name_man="Bro", name_woman="Sis"):
    print("Hello, " + name_man + " & " + name_woman + "!")
```

```
>>> hello()
```

```
Hello, Bro & Sis!
```

```
>>> hello(name_woman="Lady")
```

```
Hello, Bro & Lady!
```

```
>>> hello(name_woman="Mbakyu", name_man="Mas")
```

```
Hello, Mas & Mbakyu!
```

Імпорт

- Код, створений іншими людьми, слід використовувати повторно!
- Два способи імпорту модулів (= Python files):

- Родова форма: `import module_name`

`import math`

`print(math.sqrt(4))`

- Родова форма: `from module_name import function_name`

`from math import sqrt`

`print(sqrt(4))`

Рядок

- Рядок — це послідовність символів, наприклад «Python — це круто»
- Кожен символ має індекс

P	y	t	h	o	n		i	s		c	o	o	l
0	1	2	3	4	5	6	7	8	9	10	11	12	13

- Доступ до символу: `string[index]`
`x = "Python is cool"`
`print(x[10])`
- Доступ до підрядка через нарізку: `string[start:finish]`
`print(x[2:6])`

Операції над рядками

P	y	t	h	o	n		i	s		c	o	o	l
0	1	2	3	4	5	6	7	8	9	10	11	12	13

```
>>> x = "Python is cool"
>>> "cool" in x      # membership
>>> len(x)           # length of string x
>>> x + "?"          # concatenation
>>> x.upper()         # to upper case
>>> x.replace("c", "k") # replace characters in a string
```

Операції над рядками: розділити

P	y	t	h	o	n		i	s		c	o	o	l
0	1	2	3	4	5	6	7	8	9	10	11	12	13



`x.split(" ")`

P	y	t	h	o	n
0	1	2	3	4	5

i	s
0	1

c	o	o	l
0	1	2	3

```
>>> x = "Python is cool"
```

```
>>> x.split(" ")
```

Операції над рядками: приєднання

P	y	t	h	o	n	i	s	c	o	o	l
0	1	2	3	4	5	0	1	0	1	2	3

↓
",".join(y)

P	y	t	h	o	n	,	i	s	,	c	o	o	l
0	1	2	3	4	5	6	7	8	9	10	11	12	13

```
>>> x = "Python is cool"
```

```
>>> y = x.split(" ")
```

```
>>> ", ".join(y)
```


Вхід/Вихід

- Робота з даними значною мірою включає читання та запис!
- Дані бувають двох типів:
 - Текст: зрозумілий для людини, закодований у ASCII/UTF-8, наприклад: .txt, .csv

U+0041	A
U+0042	B
U+0043	C
U+0044	D
U+0045	E

- Двійковий код: машиночитане кодування для конкретної програми, приклад: .mp3, .mp4, .jpg 42

Вхід

```
python  
is  
cool
```

cool.txt

```
x = open("cool.txt", "r") # _read mode
```

```
y = x.read() # read the whole  
print(y)
```

```
x.close()
```

Вхід

```
x = open("cool.txt", "r")  
  
# read line by line  
for line in x:  
    line = line.replace("\n","")  
    print(line)  
  
x.close()
```

Вхід

```
python  
is  
cool
```

cool.txt

```
x = open("C:\\Users\\Fariz\\cool.txt", "r") # absolute location
```

```
for line in x:  
    line = line.replace("\n", "")  
    print(line)
```

```
x.close()
```

Вихід

write mode

```
x = open("carpe-diem.txt", "w")
```

```
x.write("carpe\ndiem\n")
```

```
x.close()
```

append mode

```
x = open("carpe-diem.txt", "a")
```

```
x.write("carpe\ndiem\n")
```

```
x.close()
```

Write mode overwrites files,

while append mode does not overwrite files but instead appends at the end of the files' content

Списки

- Якщо рядок — це послідовність символів, **то список — це послідовність елементів!**
- Список зазвичай береться в квадратні дужки []
- На відміну від рядків, де об'єкт є фіксованим (= незмінним), ми можемо вільно змінювати списки (тобто списки є змінними).

```
x = [1, 2, 3, 4]
x[0] = 4
x.append(5)
print(x) # [4, 2, 3, 4, 5]
```

Перелік операцій

```
>>> x = [ "Python", "is", "cool" ]  
>>> x.sort()          # sort elements in x  
>>> x[0:2]            # slicing  
>>> len(x)            # length of string x  
>>> x + ["!"]         # concatenation  
>>> x[2] = "hot"      # replace element at index 0 with "hot"  
>>> x.remove("Python") # remove the first occurrence of "Python"  
>>> x.pop(0)          # remove the element at index 0
```

Розуміння списку

It is basically a cool way of generating a list

[**expression** **for**-clause **condition**]

Example:

[**i*2** **for** i in [0,1,2,3,4] **if** i%2 == 0]

[**i.replace("o", "i")** **for** i in ["Python", "is", "cool"] **if** len(i) >= 3]

Кортежі

- Як і список, але ви не можете його змінити (= незмінний)
- Кортеж зазвичай (але не обов'язково) береться в круглі дужки ()
- Все, що працює зі списками, працює з кортежами, крім функцій, які змінюють вміст кортежів
- Приклад:
 `x = (0,1,2)`
 `y = 0,1,2` # same as x
 `x[0] = 2` # this gives an error

Набори

- На відміну від списків, у наборах видаляються дублікати і немає порядку елементів!
- Набір має вигляд { e1, e2, e3, ... }
- Операції включають: перетин, об'єднання, різницю.
- Приклад:

```
x = [0,1,2,0,0,1,2,2]
y = {0,1,2,0,0,1,2,2}
print(x)
print(y)
print(y & {1,2,3}) # intersection
print(y | {1,2,3}) # union
print(y - {1,2,3}) # difference
```

Словники

- Словники відображають від ключів до значень!
- Вміст у словниках не впорядкований.
- Словник має форму { k1:v1, k2:v2, k3:v3, ... }
- Приклад:

```
x = {"indonesia":"jakarta", "germany":"berlin","italy":"rome"}  
print(x["indonesia"]) # get value from key  
x["japan"] = "tokyo" # add a new key-value pair to dictionary  
print(x) # {'italy': 'rome', 'indonesia': 'jakarta', 'germany': 'berlin', 'japan': 'tokyo'}
```

Питання та відповіді